

Codex 完整聊天记录

记录范围：本轮 Codex 入门学习对话；包含用户与 Codex 的可见聊天内容，截图按出现位置插入。

说明：命令执行日志、工具内部输出不作为聊天内容收录；截图若当前文件不可读或缩放后不可辨认，则标注“截图略”。

01. 用户

我是一个想学习codex的新手，你能做我的入门引导么，教会我基本的操作并引导我实践。

02. Codex

当然可以。我可以做你的 Codex 入门教练，带你一边学一边练。刚才我尝试读取最新 Codex 官方手册，但当前环境连不上 OpenAI 文档站点，所以我先基于你现在这个 Codex 桌面环境能看到的功能来教；遇到可能随版本变化的地方，我会提醒你。

我们先这样学：

1. 认识 Codex 能做什么

Codex 可以帮你读项目、改代码、运行检查、解释报错、生成文件、做网页/应用、处理文档表格、查 GitHub、管理邮件/云盘等。你只需要说目标，不需要一开始就会命令。

2. 学会发任务

好的请求通常包含三件事：“我要什么结果” + “在哪个项目/文件里” + “有什么限制”。

例如：

练习1：帮我认识当前工作区，告诉我这里有什么文件、可能是什么项目。

我建议从练习1开始。这样你会先学会 Codex 最常用的能力：读项目、总结结构、找到入口。

03. 用户

好的

04. Codex

好，我们从“认识当前工作区”开始。我查看了你的当前工作区：它原本是空的，也不是 Git 项目。然后我创建了一个练习文件：

```
codex-practice/hello.md
```

里面现在是：

```
# Codex Practice
```

```
Hello, Codex!
```

```
This is our first practice file.
```

你刚刚已经体验了 Codex 的一个基本 workflow：查看环境 → 判断情况 → 创建/修改文件 → 读回确认。

下一步你可以继续：

练习2：帮我把这个练习文件改成中文自我介绍模板。

05. 用户

选中文本：练习1：帮我认识当前工作区，告诉我这里有什么文件、可能是什么项目。

06. Codex

你刚刚选中的这句话就是一个很好的 Codex 指令：

练习1：帮我认识当前工作区，告诉我这里有什么文件、可能是什么项目。

这类指令适合用来让 Codex 先观察，不急着想文件。

这次练习的结果是：当前工作区原本几乎是空的，不像一个已有项目；我后来为练习创建了一个文件：

codex-practice/hello.md

所以它现在更像是一个“Codex 入门练习区”，不是正式软件项目。

07. 用户

选中文本：练习2：帮我把这个练习文件改成中文自我介绍模板。

08. Codex

练习2完成了。我已经把这个文件改成中文自我介绍模板：

codex-practice/hello.md

这次你练到的是 Codex 的第二个核心能力：根据自然语言修改文件，并在修改后读回确认。

你可以继续下一课：

练习3：帮我创建一个简单网页，展示这份自我介绍。

09. 用户

选中文本：练习3：帮我创建一个简单网页，展示这份自我介绍。

10. Codex

练习3完成了：我已经创建了一个简单网页，用来展示你的自我介绍。

文件在这里：

codex-practice/index.html

这次你练到的是 Codex

的第三个核心能力：把一份文字内容转换成可打开的网页作品。这个页面不需要启动服务，直接打开 index.html 就能看。

下一课可以继续：

练习4：帮我给这个网页加一个按钮，点击后显示一句欢迎语。

11. 用户

选中文本：练习4：帮我给这个网页加一个按钮，点击后显示一句欢迎语。

12. Codex

练习4完成了。我已经给网页加了一个按钮：点击“显示欢迎语”后，会出现：

欢迎你，王维！这是你用 Codex 做出的第一个网页互动。

修改的文件是：

codex-practice/index.html

这次你练到的是 Codex

的第四个核心能力：给已有网页添加交互功能。也就是不仅能“展示内容”，还可以“响应用户操作”。

下一课可以继续：

练习5：帮我解释这个网页的 HTML、CSS、JavaScript 分别负责什么。

13. 用户

选中文本：练习5：帮我解释这个网页的 HTML、CSS、JavaScript 分别负责什么。

14. Codex

练习5开始：这个网页由三部分组成，分别是 HTML、CSS、JavaScript。

HTML：负责内容和结构。HTML 决定网页上“有什么”。例如按钮这一行：

```
<button type="button" id="welcomeButton">显示欢迎语</button>
```

意思是：页面上放一个按钮，按钮文字是“显示欢迎语”。

CSS：负责样式和布局。CSS 决定网页“长什么样”。它控制背景颜色、文字颜色、页面宽度、顶部区域布局、头像圆形样式、按钮样式和手机屏幕适配。

JavaScript：负责互动。JavaScript

决定网页“会做什么”。核心逻辑是：当用户点击按钮时，把欢迎语写到页面上。

一句话总结：

HTML 放内容，CSS 管好看，JavaScript 管互动。

下一课可以继续：

练习6：帮我把欢迎语改成可以输入名字后自动生成。

15. 用户

我无法找到一个可以查看html背后的代码，是不是没有教我如何使用vscode

16. Codex

对，你说得很对：我前面教了“让 Codex 创建网页”，但还没有教你在哪里查看 HTML 背后的代码。这是入门里很关键的一步。

你现在看到的是浏览器预览页面，它只负责展示网页效果；真正的代码在这个文件里：

codex-practice/index.html

方式1：用 VS Code 打开

1. 打开 VS Code。
2. 点左上角 File / 文件。
3. 点 Open Folder / 打开文件夹。
4. 选择 C:\Users\HUAWEI\Documents\codex-restart。
5. 左侧会看到 codex-practice 文件夹。
6. 点开 index.html，就能看到网页代码。

方式2：用记事本打开

找到 index.html，右键选择“打开方式”，选记事本或 VS Code。

你现在应该知道的关系：

index.html = 网页代码文件

浏览器 = 查看网页效果

VS Code = 查看和编辑代码

Codex = 帮你理解、修改、检查代码

17. 用户

选中文本：练习6：教我如何用 VS Code 打开这个项目并认识左侧文件列表。

18. Codex

好，练习6我们补上非常重要的一环：用 VS Code 打开项目，看见代码文件。

第1步：打开 VS Code。

第2步：打开项目文件夹，选择 C:\Users\HUAWEI\Documents\codex-restart。

注意：选择的是 codex-restart 这个文件夹，不是单独选择 index.html。

第3步：认识左侧文件列表。打开后，VS Code 左边会有：

```
codex-restart
├── codex-practice
├── hello.md
└── index.html
```

第4步：打开 HTML 代码，点 codex-practice > index.html。

第5步：形成基本认知：

浏览器：看网页效果

VS Code：看和改代码

Codex：帮你理解、修改、检查代码

19. 用户

好的，我看到了html，是否应该还要有单独的style.css和script.js，又或者都集成在html里面了

20. Codex

你理解得很准确：现在这个网页是把 CSS 和 JavaScript 都集成在 index.html 里面了。

目前结构是：

```
index.html
```

里面同时包含三部分：

<style> CSS 样式写在这里 </style>

<body> HTML 页面内容写在这里 </body>

<script> JavaScript 互动逻辑写在这里 </script>

这样做适合新手练习，因为只有一个文件，打开和理解都比较简单。

但实际项目里，通常会拆成三个文件：

```
codex-practice
├── index.html
├── style.css
└── script.js
```

小练习、临时页面：可以都写在 HTML 里。

稍微正式一点的网页：建议拆成 style.css 和 script.js。

21. 用户

选中文本：练习7：帮我把 index.html 里的 CSS 和 JavaScript 拆分成 style.css 和 script.js。

22. Codex

练习7完成了。现在你的网页已经从一个 HTML 文件，拆成了更标准的三文件结构：

```
codex-practice
├── hello.md
├── index.html
├── style.css
└── script.js
```

对应关系是：

index.html：页面内容和结构

style.css：颜色、布局、按钮、卡片等样式

script.js：点击按钮后显示欢迎语的互动逻辑

index.html 现在通过这两行连接外部文件：

```
<link rel="stylesheet" href="style.css">
```

```
<script src="script.js"></script>
```

你可以在 VS Code 左侧分别点开这三个文件看。浏览器里刷新页面后，样式和按钮互动应该还和之前一样。

23. 用户

选中文本：练习7：教我在 VS Code

里手动修改按钮文字，然后刷新浏览器查看效果。另选中文本：练习6：教我如何用 VS Code

打开这个项目并认识左侧文件列表。

24. Codex

可以，我们把这个当成“手动练习课”：这次我不替你改文件，你自己在 VS Code 里改，我负责带路。

目标：把网页按钮文字从“显示欢迎语”改成“点我看看”。

第1步：在 VS Code 里找到文件 codex-practice > index.html。

第2步：按 Ctrl + F 搜索“显示欢迎语”。

第3步：把按钮行改成：

```
<button type="button" id="welcomeButton">点我看看</button>
```

注意：只改中间的中文文字，其他内容先不要动。

第4步：按 Ctrl + S 保存。

第5步：回到网页按 F5 刷新。

这节课你学到的是：

VS Code 里改代码

保存文件

浏览器刷新看效果

这是做网页最基础、最常用的循环：改一下 → 保存 → 刷新 → 看结果。

25. 用户

很好，关于项目上传到github呢，如何操作

26. Codex

把项目上传到 GitHub，通常分两种方式：用 VS Code 图形界面，或者用命令行。你是新手，我建议先学 VS Code 图形界面。

当前这个文件夹还不是 Git 项目，所以完整流程是：

本地文件夹 → 初始化 Git → 第一次提交 → 创建 GitHub 仓库 → 推送到 GitHub

方式一：用 VS Code 上传，推荐：

1. 在 VS Code 打开 C:\Users\HUAWEI\Documents\codex-restart。
2. 点击左侧源代码管理图标。
3. 如果看到“初始化仓库”，点它。
4. 你会看到文件列表变化。
5. 在消息框里输入第一次提交说明，例如：创建 Codex 入门练习网页。
6. 点击提交。

7. 登录 GitHub。

8. 找到“发布分支”或 Publish Branch，点击后 VS Code 会帮你创建仓库并上传代码。

方式二：命令行：

```
git init
git add .
git commit -m "创建 Codex 入门练习网页"
git branch -M main
git remote add origin 你的GitHub仓库地址
git push -u origin main
```

建议下一步：练习8：带我在 VS Code 里初始化 Git 仓库，并完成第一次提交。

27. 用户

截图：VS Code 源代码管理页面，显示“当前打开的文件夹中没有 Git 存储库”，有“初始化仓库”和“发布到 GitHub”按钮。

用户截图：



28. Codex

你现在看到的这个界面非常对，说明 VS Code 已经识别到：当前文件夹还没有 Git 仓库。

这里有两个按钮：

初始化仓库

发布到 GitHub

我建议你先点“初始化仓库”。

原因：

初始化仓库 = 先在电脑本地建立版本管理

发布到 GitHub = 直接上传到 GitHub

对新手来说，最好按顺序来：

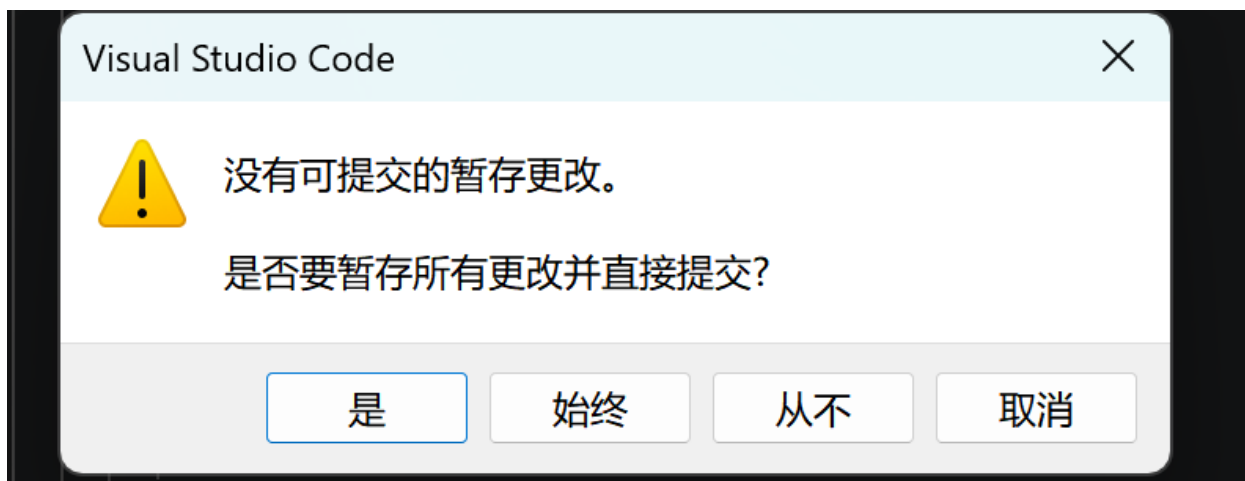
初始化仓库 → 第一次提交 → 发布到 GitHub

点击“初始化仓库”后，左侧应该会出现一批“更改的文件”。

29. 用户

截图：VS Code 弹窗提示“没有可提交的暂存更改。是否要暂存所有更改并直接提交？”

用户截图：



30. Codex

这个弹窗很正常，意思是：

你还没有暂存文件。要不要先把所有更改暂存起来，然后直接提交？

你现在可以点：是。

点“是”之后，VS Code 会帮你完成两步：

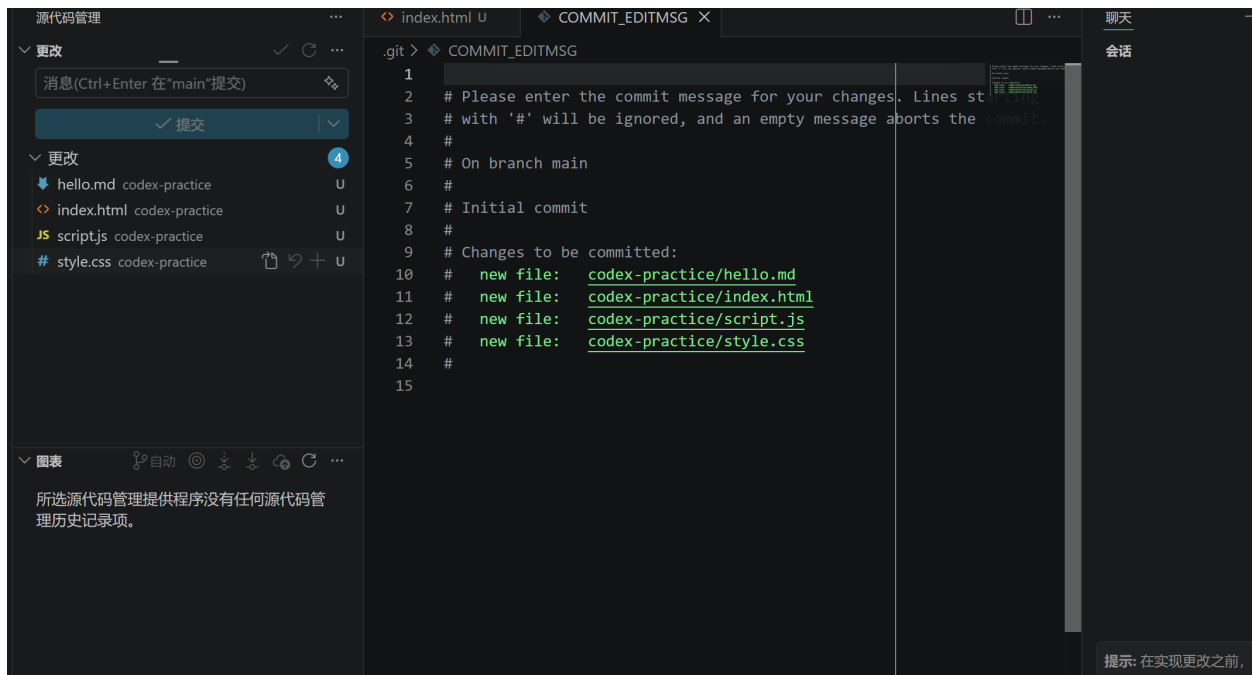
暂存所有文件 → 提交

如果以后想少弹这种提示，也可以手动点文件旁边的 + 号先暂存，再点提交。

31. 用户

截图：VS Code 打开 COMMIT_EDITMSG，左侧显示 4 个 U 状态文件：hello.md、index.html、script.js、style.css。

用户截图：



32. Codex

你现在进入的是提交信息编辑页面。这说明 VS Code 正在等你填写这次提交的说明。

中间这个文件叫 COMMIT_EDITMSG。

你只需要在第1行空白处输入提交说明，例如：

创建 Codex 入门练习网页

注意：不要写在带 # 的行里，因为 # 开头的内容会被 Git 忽略。

然后：

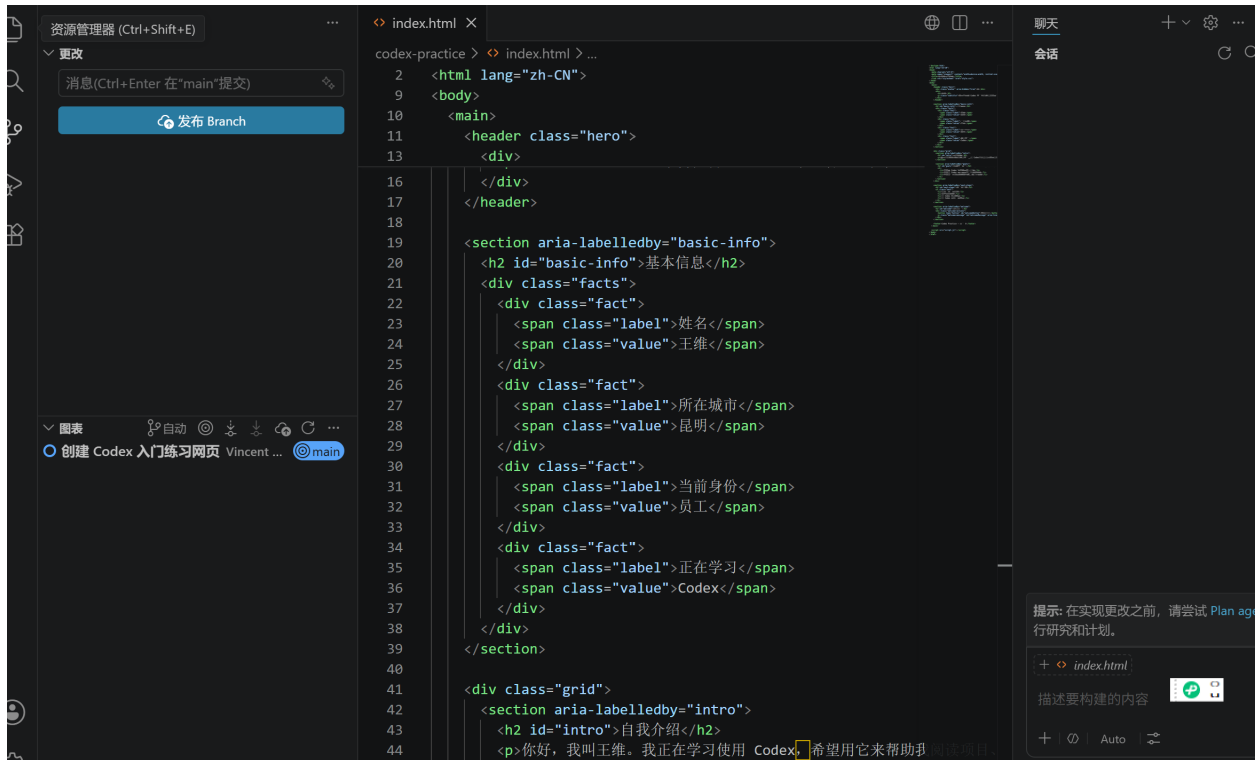
1. 按 Ctrl + S 保存。
2. 关闭这个 COMMIT_EDITMSG 标签页。
3. VS Code 就会继续完成提交。

左边文件前面的 U 意思是 Untracked，还没被 Git 正式记录过的新文件。这是第一次提交时很正常的状态。

33. 用户

截图：VS Code 源代码管理显示“发布 Branch”，图表里出现“创建 Codex 入门练习网页”。

用户截图：



34. Codex

很好，你已经完成了本地第一次提交。

我确认当前状态是：main 分支是干净的，没有未提交改动。

也就是说你已经完成：

初始化仓库 → 第一次提交

现在下一步就是：

发布到 GitHub

你可以点击左侧蓝色按钮“发布 Branch”。它的意思是：把你本地的 main 分支发布到 GitHub。

点了之后，VS Code 可能会让你选择 Public Repository 或 Private Repository。建议新手先选 Private Repository。

35. 用户

刚出现时了两个对话框，一个是打开github，一个问是否定期……，但是对话框消失了，接下来我该如何操作

36. Codex

不用担心，弹窗虽然消失了，但事情已经成功了。

我帮你确认了一下：你的项目已经连接到了 GitHub，并且第一次提交已经上传成功。

GitHub 仓库地址是：

<https://github.com/why911-1/codex-restart>

当前状态是：

本地 main 分支 = GitHub 上的 main 分支

也就是说你已经完成：

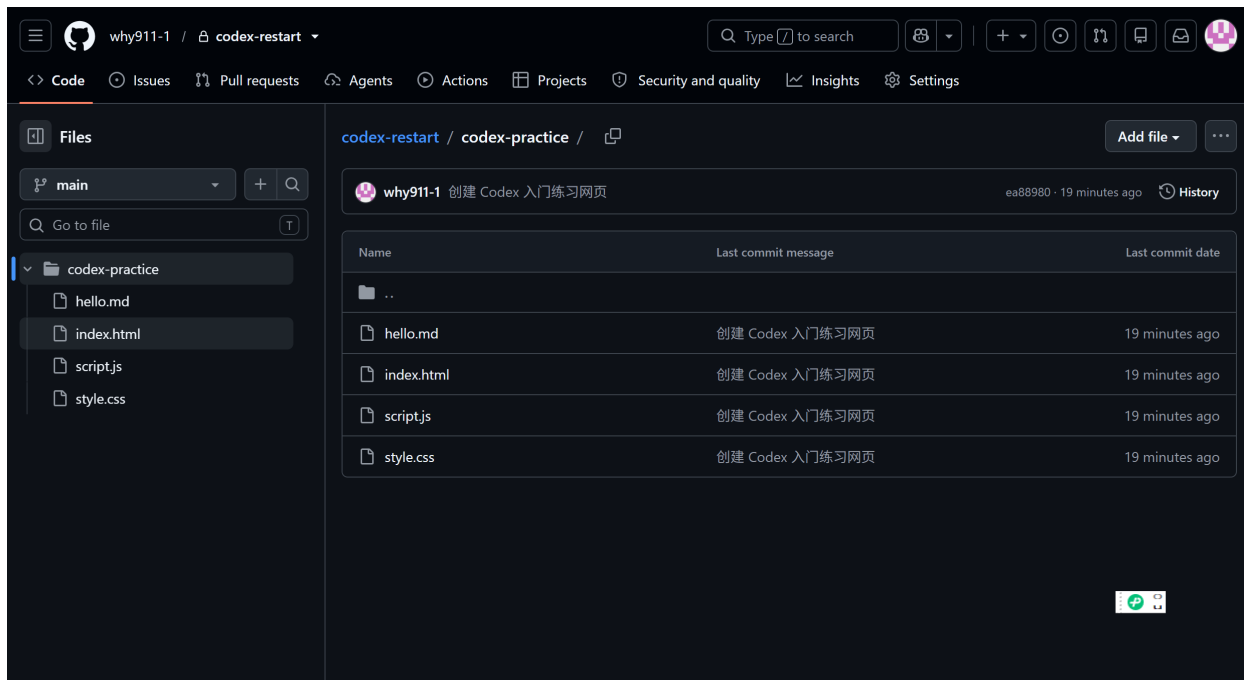
初始化仓库 → 第一次提交 → 发布到 GitHub

刚才那个“是否定期……”的弹窗大概率是 VS Code 询问是否定期运行 GitHub 相关操作或同步检查。它消失了没关系，不影响上传结果。

37. 用户

截图：GitHub 仓库 why911-1 / codex-restart, codex-practice 文件夹内包含 hello.md、index.html、script.js、style.css。

用户截图：



38. Codex

太好了，这张图说明你已经成功上传到 GitHub 了。

你现在看到的是：

why911-1 / codex-restart

里面有：

codex-practice

- ├─ hello.md
- ├─ index.html
- ├─ script.js
- └─ style.css

这就表示你的本地项目已经变成了 GitHub 上的一个仓库。

你现在已经完成了整条流程：

创建网页 → 初始化 Git → 提交版本 → 发布到 GitHub

以后最常用的循环：

修改代码 → 保存 → 提交 commit → 同步 / push 到 GitHub

39. 用户

选中文本：练习9：修改网页内容后，再提交一次并同步到 GitHub。问题：但是我如何确定他出现在我的private里面呢

40. Codex

可以确定。你截图里仓库名旁边有一个小锁图标：

why911-1 /  codex-restart

这个锁就表示：这是 Private 私有仓库。

还可以在 Settings 里查看仓库可见性。简单记：

有锁 = Private 私有

没锁 = Public 公开

所以你现在这个 codex-restart 仓库是私有的。

41. 用户

选中文本：练习9：修改网页内容后，再提交一次并同步到 GitHub。

42. Codex

我已经完成了“修改网页内容”这一步：把页脚改成了：

Codex Practice - 练习 9：再次提交并同步

文件是：

codex-practice/index.html

但我这边没法直接替你提交，因为当前 Git 元数据被 VS Code/Windows 拒绝写入。没关系，这正好进入练习重点：你在 VS Code 里手动提交并同步。

步骤：

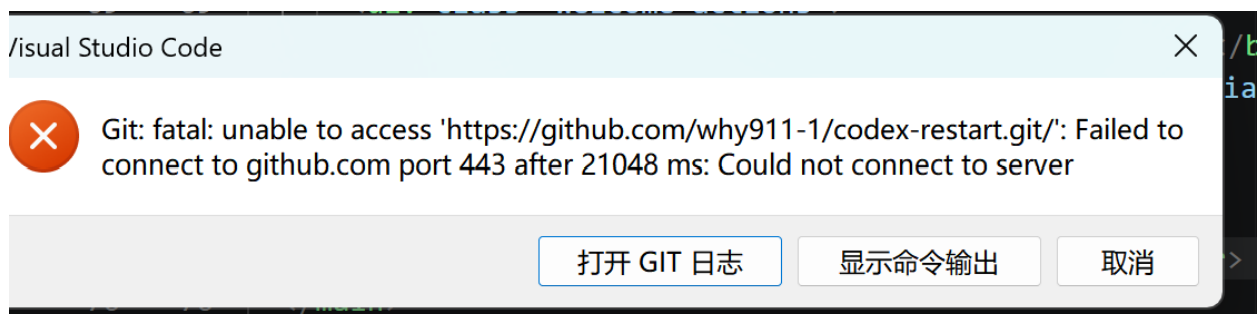
1. 打开 VS Code 左侧源代码管理。
2. 你应该会看到 index.html 有一个修改标记。
3. 在提交消息框输入：更新练习9页脚说明
4. 点击提交。
5. 提交成功后，点击同步更改或 Sync Changes。

完成后去 GitHub 刷新页面，你会看到 index.html 的提交信息变成这次新的说明。

43. 用户

截图：VS Code 弹窗报错：Git: fatal: unable to access 'https://github.com/why911-1/codex-restart.git/': Failed to connect to github.com port 443。

用户截图：



44. Codex

这个报错不是你操作错了，而是 VS Code 现在连不上 GitHub。

关键句是：

Failed to connect to github.com port 443

Could not connect to server

意思是：本地提交可能已经成功了，但同步到 GitHub 时网络连接失败。

你现在先点取消，然后：

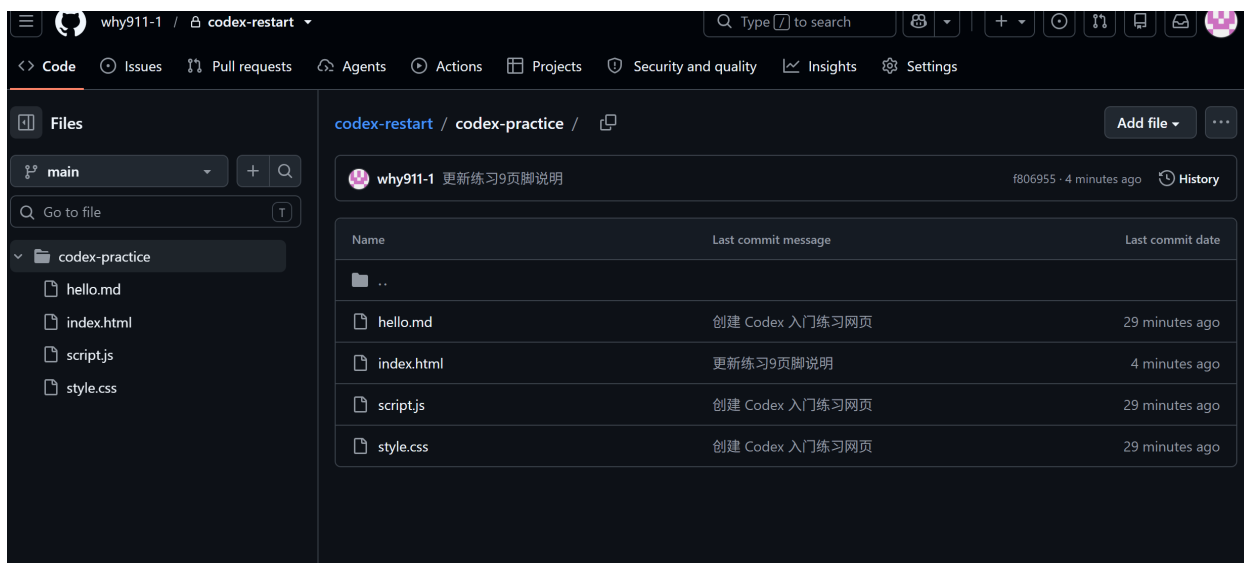
1. 确认浏览器能不能打开 <https://github.com>。
2. 如果打不开，说明当前网络访问 GitHub 有问题，可以换网络、开启可用代理，或稍后再试。
3. 如果浏览器能打开 GitHub，回到 VS Code 源代码管理，重新点“同步更改”或“Push / 推送”。

如果出现“同步更改 1”，意思是本地已经提交了 1 个版本，但还没成功上传到 GitHub。这是可恢复的，不会丢。

45. 用户

截图：GitHub 上 index.html 的提交信息已变为“更新练习9页脚说明”。

用户截图：



46. Codex

成功了。你这张截图已经证明练习9的修改同步到 GitHub 了。

证据有两个：

1. 顶部最新提交信息变成了：更新练习9页脚说明。
2. index.html 这一行的提交信息也是：更新练习9页脚说明。

这说明你刚刚完成了完整流程：

修改网页 → 本地提交 commit → 同步 push 到 GitHub → GitHub 页面显示最新版本。

以后每次改项目，基本都按这个循环：

1. 改文件
2. 保存
3. 查看源代码管理里的变化
4. 写提交说明
5. 提交
6. 同步到 GitHub
7. 刷新 GitHub 确认

47. 用户

选中文本：练习10：我如何把 GitHub 上的网页变成一个可以访问的网址。

48. Codex

这一步叫 GitHub Pages：把仓库里的静态网页发布成网址。

但你这里有一个重点：你的仓库是 Private 私有仓库。GitHub Pages 对公开仓库可用；私有仓库发布 Pages 通常需要 GitHub Pro / Team / Enterprise 等付费计划。另外，Pages 站点一旦发布，是公开可访问的，即使仓库本身是私有的。

操作：

1. 打开 GitHub 仓库：<https://github.com/why911-1/codex-restart>
2. 点击 Settings。
3. 左侧找到 Pages。
4. 在 Build and deployment 里设置：

Source: Deploy from a branch

Branch: main

Folder: / root

5. 点击 Save。

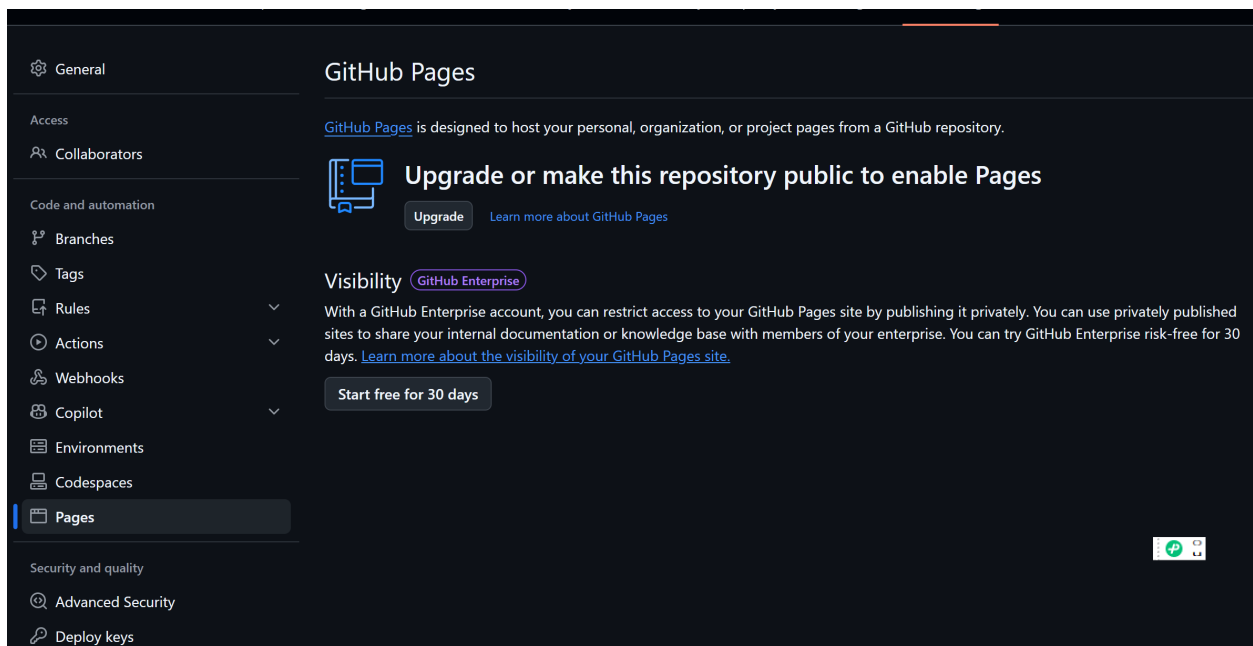
等几分钟，GitHub 会生成网址。因为 index.html 在 codex-practice 文件夹里，所以真正访问地址更可能是：
<https://why911-1.github.io/codex-restart/codex-practice/>

如果 GitHub 提示私有仓库不能启用 Pages，可以选择把仓库改成 Public，或者保持 Private 不发布网页。

49. 用户

截图：GitHub Pages 页面提示 Upgrade or make this repository public to enable Pages。

用户截图：



50. Codex

你现在看到的提示正好验证了前面说的限制：

Upgrade or make this repository public to enable Pages

意思是：

要启用 GitHub Pages，要么升级付费，要么把这个仓库改成公开。

两个选择：

选择 A：保持 Private。优点是安全，只有你能看代码；缺点是不能免费发布成网页网址。

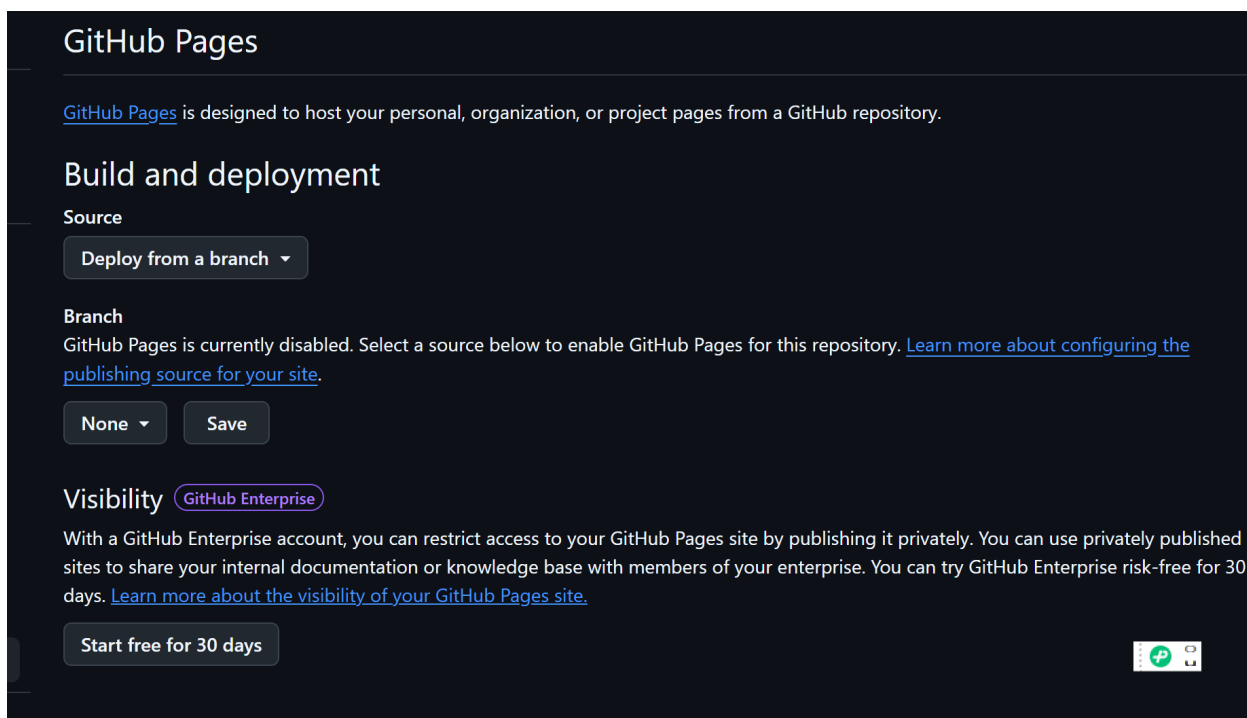
选择 B：改成 Public。优点是可以免费开启 GitHub Pages；缺点是所有人都可以看到仓库代码。

如果想继续练习 GitHub Pages，需要先把仓库改成 Public，然后回到 Settings → Pages 选择 Deploy from a branch、main、/root。

51. 用户

截图：GitHub Pages 设置页，Source 为 Deploy from a branch，Branch 仍为 None。

用户截图：



52. Codex

很好，现在你已经进入可以设置 GitHub Pages 的页面了。

请这样选：

1. 在 Branch 下面，把 None 改成 main。
2. 选完 main 后，旁边可能会出现文件夹选项，选择 / root。
3. 点击 Save。

设置完成后，GitHub 会开始部署。通常等 1 到 3 分钟，页面上方会出现一个访问地址。

因为你的网页在 codex-practice 文件夹里，所以最后访问地址大概是：

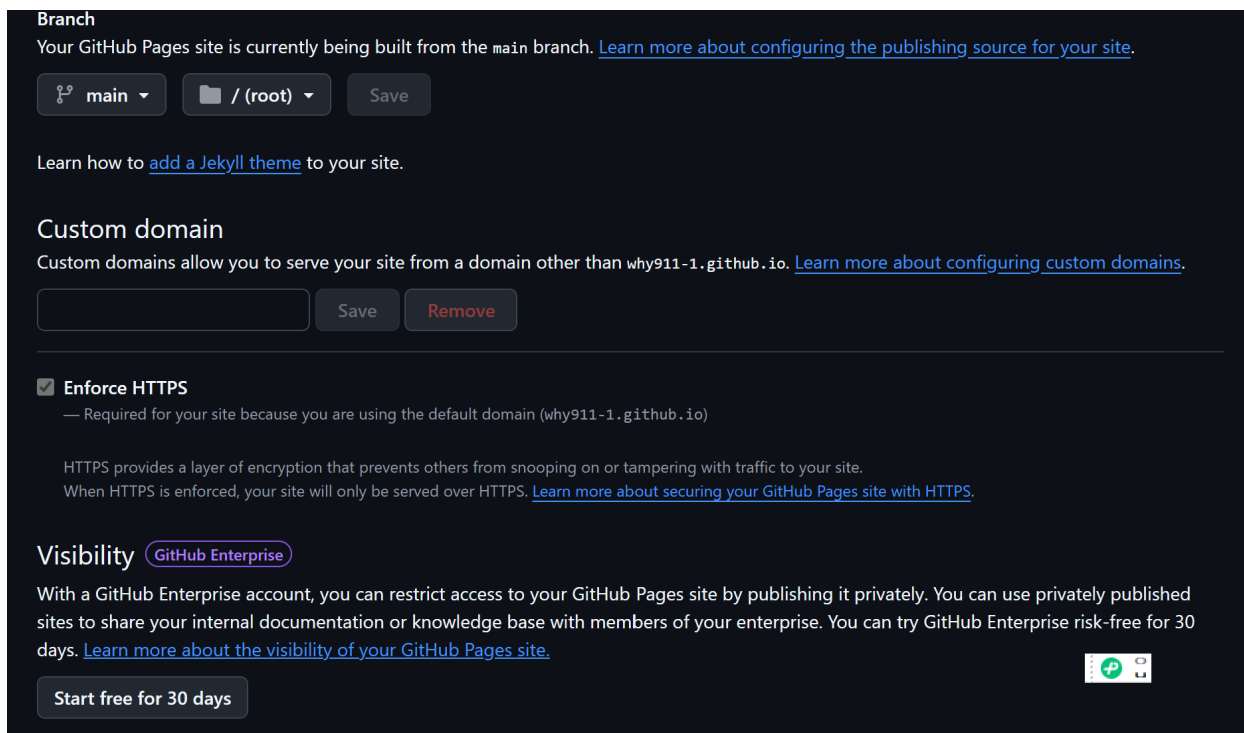
`https://why911-1.github.io/codex-restart/codex-practice/`

注意：如果只访问 `https://why911-1.github.io/codex-restart/` 可能会显示 404，因为仓库根目录下没有 index.html。

53. 用户

截图：GitHub Pages 显示 Your GitHub Pages site is currently being built from the main branch，Branch 为 main，Folder 为 /root。

用户截图：



54. Codex

现在设置已经成功了。页面上这句话很关键：

Your GitHub Pages site is currently being built from the main branch.

意思是：

GitHub 正在从 main 分支构建你的网页。

接下来等待 1 到 3 分钟，然后访问：

`https://why911-1.github.io/codex-restart/codex-practice/`

如果第一次打开是 404，不要慌，通常只是还没构建完成。等一会儿刷新即可。

因为你选择的是 main / (root)，GitHub Pages 会从仓库根目录发布。真正的 index.html 在 codex-practice 里面，所以访问时要带 /codex-practice/。

55. 用户

成功了，今天就到这里吧，明天带我继续学习。

56. Codex

太好了，今天这条线走得很完整：

创建网页

- 拆分 HTML/CSS/JS
- 用 VS Code 查看和修改
- 初始化 Git
- 提交到 GitHub
- 开启 GitHub Pages
- 得到公开访问网址

你现在已经完成了一个真正从本地到上线的小作品：

`https://why911-1.github.io/codex-restart/codex-practice/`

明天我们可以继续从这里往下学，比如：

练习11：给网页增加一个输入框，让用户输入名字后生成欢迎语。

今天先收工。你已经把最关键的第一圈跑通了。

57. 用户

先把今天的所有对话用 pdf 插件里面。

58. Codex

已整理成 PDF，放在：

output/pdf/codex-learning-log-2026-06-29. pdf

内容包括今天的学习主线、每个练习、Git/GitHub/GitHub Pages 的操作记录、遇到的问题，以及明天建议继续的练习。

我做了文本检查：PDF 共 3 页，关键内容和最终网址都已写入。视觉渲染检查没有完成，因为当前环境缺少 PDF 渲染工具。

59. 用户

截图：打开 PDF 后中文显示为大量问号。

用户截图：



60. Codex

你说得对，上一版 PDF 中文乱码了。我已经重新生成并做了页面预览检查，现在中文显示正常。

新的 PDF 还是这个文件：

output/pdf/codex-learning-log-2026-06-29. pdf

我检查了 3 页预览：标题、表格、正文、网址和结尾都正常，没有问号乱码。

61. 用户

非常好，接下来将这个对话里面的所有聊天记录用 pdf 插件进行记录，不是生成简要的学习报告，而是全部的聊天记录。如果是我给你的截图，能清晰的呈现就一并呈现，如果无法清晰呈现，就显示“截图略”。我需要保存下来反复学习。

62. Codex

收到。本 PDF

将按完整聊天记录方式保存，不再做简要学习报告；用户截图会按出现位置插入，无法清晰呈现时标注“截图略”。